Лабораторная работа 1 по Машинному обучению

Введение в машинное обучение

Задания

Часть A

1. Пусть у нас есть два вектора: $a = \begin{bmatrix} 1 \\ 5 \\ -3 \\ 2 \end{bmatrix}$ и $b = \begin{bmatrix} 8 \\ 2 \\ 4 \\ 7 \end{bmatrix}$

   (a) Вычислите $b - a$ и $2a^T + 3b^T$.

   (b) Вычислите $a^T b$. Как называется эта операция?

2. Пусть матрица $M = \begin{bmatrix} 1 & 2 & 4 \\ -2 & 5 & -1 \end{bmatrix}$ и векторы $v = \begin{bmatrix} 3 \\ 0 \\ -5 \end{bmatrix}$ и $w = \begin{bmatrix} 2 \\ -3 \end{bmatrix}$.

   (a) Посмотрите на эти выражения: $Mv, Mv^T, vM, v^T M, w^T M, wM$. Какие из этих операций являются корректными и почему? Вычислите их.

   (b) Повторите это для выражений $vw, vw^T, v^T w, wv, wv^T$.

3. Пусть f — это функция, которая принимает вектор $v = \begin{bmatrix} v_1 & v_2 & \dots & v_n \end{bmatrix}$ и возвращает скалярное значение $1 \cdot v_1 + 2 \cdot v_2 + \dots + n \cdot v_n$.

   (a) Каким типом объекта является $\frac{\partial f(v)}{\partial v_3}$? Вычислите.

   (b) Каким типом объекта является $\nabla_v f(v)$? Вычислите.

Часть B

4. Создайте пустую директорию и откройте её через VS Code. Создайте виртуальное окружение и установите пакет numpy:

```
python -m venv venv
source venv/bin/activate  # macOS/Linux
venv\Scripts\activate     # Windows
pip install numpy
```

   Далее в начале кода в Python прописывайте `import numpy as np`.

   У numpy есть хорошая документация:

   `https://numpy.org/doc/stable/user/quickstart.html`.

   Мы будем вектора создавать как **2-мерный** массив `np.array`!

   Функции и методы, которые пригодятся в этой лабе:

```
np.transpose    #траноспонирование матрицы
np.T    #синоним для transpose
np.array.shape    #размеры матрицы
np.dot    #умножение матриц
np.linalg.norm    #возвращает норму матрицы
```

(!) В этой лабе нельзя пользоваться циклами!

(a) Повторите задания 1 и 2 в программе (создайте вектора и матрицы в виде 2-мерных `np.array`, ответы выводите в консоль). Убедитесь, что ваши ручные ответы совпадают с результатами программы.

(b) Напишите функцию `normalize(v)`, которая принимает вектор `v` (который является вектором-столбцом в виде 2-мерного np.array) и возвращает этот же вектор в нормализованном виде. Вектор нормализован, если у него длина (norm) равна 1.

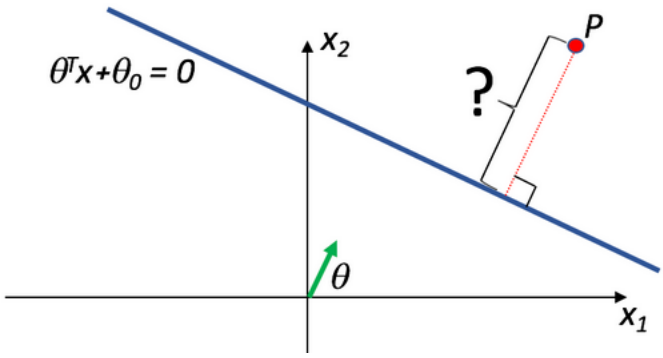
(c) Пусть у вас есть данные про рост и вес четырех людей: 170 см и 65 кг, 174 см и 80 кг, 179 см и 90 кг, 166 см и 62 кг. Представьте эти данные в виде `np.array`, в которой каждый человек — это одна строкар размера 2 (его рост и вес).

Напишите функцию `calc`, которая будет принимать такую матрицу и возвращать матрицу размера 1×2 — средний рост и средний вес людей.

Это нужно будет сделать через операции с матрицами — никаких циклов!

Часть C

5. Посмотрите на это изображение:



(a) Напишите функцию, которая `signed_dist(x, th, th0)`, которая находит *расстояние со знаком* от гиперплоскости $\theta^T x + \theta_0 = 0$ до точки x.

Здесь `x` и `th` — это векторы-столбцы одинакового размера, а `th0` — это скаляр (т.е. просто число).

Подсказка: расстояние вычисляется по формуле $\frac{|\theta^T x + \theta_0|}{\|\theta\|}$, где $\|\theta\|$ — это «норма» (длина) вектора θ.

(b) Напишите функцию `positive(x, th, th0)`, которая возвращает 1, если x находится в положительной стороне от гиперплоскости $\theta^T x + \theta_0 = 0$, возвращает 0, если x лежит ровно на ней, и -1, если лежит в отрицательную сторону.

(c) Далее переменная `data` — это массив $d \times n$ — датасет, представляющий из себя n объектов размерности d;

переменная `labels` — это массив $1 \times n$ — массив «меток» (значений), соответствующий датасету.

Иными словами, `data` и `labels` представляют помеченные данные (датасет, используемый в *обучении с учителем*).

Напишите функцию `score(data, labels, th, th0)`, которая возвращает для скольки объектов из датасета `(data, labels)` лейбл совпадает со знаком объекта согласно **разделяющей гиперплоскости (separating hyperplane)** `(th, th0)`.

Иными словами, функция `score` вычисляет, насколько хорошо линейный классификатор $h(x; \theta, \theta_0) = sign(\theta^T x + \theta_0)$ классифицирует заданную обучающую выборку.

(d) Нарисуйте на бумаге (можно и на компьютере) конкретный пример — не менее 7 точек на координатной плоскости, каждая помеченная плюсом или минусом. Проведите две разные разделяющие прямые вида $\theta x + \theta_0 = 0$ (лучше сначала придумайте конкретные значения θ и θ_0 и нарисуйте по ним). Вручную посчитайте *score* каждой прямой.

Нарисованные данные введите в свой код и убедитесь, что ответы вашей функции для ваших прямых совпадают с вашими «ручными» ответами.

(e) Напишите функцию `best_separator(data, labels, ths, th0s)`, которая принимает датасет `(data, labels)` и набор разделяющих прямых `(ths, th0s)`.

`ths` — массив $d \times m$, `th0s` — массив $1 \times m$. Вместе они задают m разделяющих прямых.

Функция должна среди заданных разделяющих прямых найти `(th, th0)` *лучшую* — с самым высоким `score`, и вернуть её в виде `(th, th0)` (объект типа `tuple`), где `th` — массив $d \times 1$, а `th0` — скаляр.